BlockPilot: Instance-Adaptive Policy Learning for Diffusion-based Speculative Decoding

Hao Zhang, Yiming Hu, Yong Wang, Mingqiao Mo, Xin Xiao, Xiangxiang Chu

ABSTRACT

Speculative decoding accelerates inference by using a lightweight draft model to generate candidate tokens in parallel, and are then verified by the target model, enabling lossless acceleration. Recently, diffusion-based speculative decoding further improves parallelism by generating multiple tokens per forward pass via block-level diffusion, achieving state-of-the-art (SOTA) performance. However, existing methods adopt a fixed inference block size and assume a uniform optimal decoding strategy across all inputs. In this paper, we show that this assumption is suboptimal, as the optimal block size varies across samples and plays a critical role in speculative decoding performance. Moreover, these values exhibit a clear local structure, concentrating around the training block size, which reduces the problem to a low-dimensional and structured decision space. Based on these insights, we propose BlockPilot, a sample-adaptive policy that predicts the optimal block size from the prefilling representation. Specifically, we formulate block size selection as a lightweight policy learning problem and propose an instance-adaptive decision mechanism that predicts the optimal block size based on the representation of the prefilling stage. The prediction is performed only once after prefilling, allowing for seamless integration. Extensive experiments demonstrate that our method is plug-and-play, introduces minimal overhead, and consistently improves efficiency, achieving an acceptance length of 5.92 and a 4.20$\times$ speedup on Qwen3-4B under temperature $T=1$.

About this document

This is a **Reviewed Preprint**: a third-party paper that llmXive has *auto-reviewed* (with its LLM review panel) but *never modified*. The original manuscript follows this cover page **exactly as published** by its authors — llmXive re-typesets nothing and claims no authorship.
Provenance. Ingested by llmXive on 2026-07-04 — scraped from arXiv; via llmXive dashboard, GitHub issue #463; submitted by github-actions[bot].
Source. https://arxiv.org/abs/2606.31315
About llmXive. llmXive is an automated platform for open scientific discovery. Its specialist

LLM agents advance their own ideas from brainstorm to peer-reviewed paper, and they also review notable third-party papers — drawn from the most-downloaded papers on Hugging Face and from submissions by human contributors. Every submitted paper is reviewed by the LLM panel *and* used to seed a new llmXive follow-up study. This paper is one such third-party work: llmXive reviewed it but did not write it. Learn more at `https://github.com/ContextLab/llmXive`.

Automated review. See the accompanying `peer-review-llmxive.pdf` for the full reviewer report.

llmXive follow-up. A separate llmXive study building on this work: `PROJ-986-llmxive-follow-up-extending-blockpilot-i`.

arXiv:2606.31315v1 [cs.CL] 30 Jun 2026

BlockPilot: Instance-Adaptive Policy Learning for Diffusion-based Speculative Decoding

Hao Zhang **Yiming Hu**† **Yong Wang**† **Mingqiao Mo** **Xin Xiao** **Xiangxiang Chu**

AMAP, Alibaba Group
https://github.com/AMAP-ML/BlockPilot

Abstract

Speculative decoding accelerates inference by using a lightweight draft model to generate candidate tokens in parallel, and are then verified by the target model, enabling lossless acceleration. Recently, diffusion-based speculative decoding further improves parallelism by generating multiple tokens per forward pass via block-level diffusion, achieving state-of-the-art (SOTA) performance. However, existing methods adopt a fixed inference block size and assume a uniform optimal decoding strategy across all inputs. In this paper, we show that this assumption is suboptimal, as the optimal block size varies across samples and plays a critical role in speculative decoding performance. Moreover, these values exhibit a clear local structure, concentrating around the training block size, which reduces the problem to a low-dimensional and structured decision space. Based on these insights, we propose BlockPilot, a sample-adaptive policy that predicts the optimal block size from the prefilling representation. Specifically, we formulate block size selection as a lightweight policy learning problem and propose an instance-adaptive decision mechanism that predicts the optimal block size based on the representation of the prefilling stage. The prediction is performed only once after prefilling, allowing for seamless integration. Extensive experiments demonstrate that our method is plug-and-play, introduces minimal overhead, and consistently improves efficiency, achieving an acceptance length of 5.92 and a 4.20× speedup on Qwen3-4B under temperature $T = 1$.

1 Introduction

Large Language Models (LLMs) [36, 40, 11] have achieved remarkable performance across a wide range of tasks [1, 14], demonstrating strong capabilities in reasoning, code generation, and open-ended dialogue. Despite these advances, their inference efficiency is still fundamentally constrained by token-by-token autoregressive decoding [35, 28, 37]. Since each token must be generated conditioned on previously produced tokens, the decoding process is inherently sequential, leading to high latency and limited parallelism, especially for long-form generation. To alleviate this bottleneck, speculative decoding [19, 23, 21, 22, 5] introduces a lightweight draft model to propose multiple candidate tokens ahead of the target model. These candidates are then verified by the target model in parallel, allowing multiple tokens to be accepted within a single

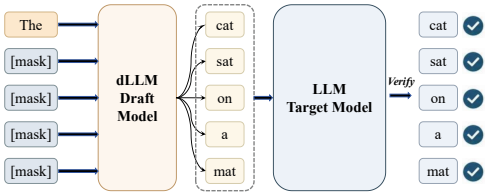


Figure 1: Diffusion-based speculative decoding with a dLLM draft model. The dLLM proposes a block of tokens in parallel, while the target LLM verifies the block and accepts the longest consistent prefix.

† Project lead and corresponding author.

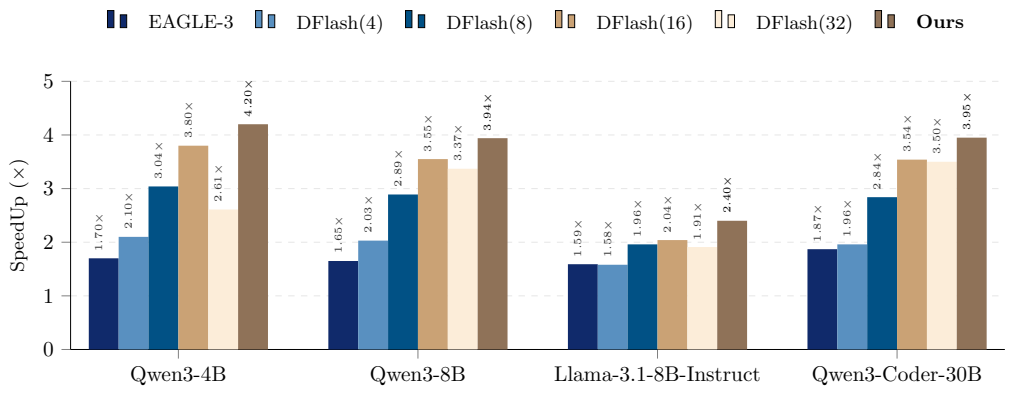


Figure 2: Speedup comparison across models under temperature $T = 1$. Our method achieves the highest acceleration across all settings. Here, DFlash(n) denotes DFlash with block size n.

decoding step. As a result, speculative decoding enables lossless acceleration without altering the output distribution of the target model.

In recent years, diffusion-based language models (dLLMs) [27, 2, 39, 10] have emerged as a promising direction, and diffusion-driven speculative decoding further improves parallelism. By using a dLLM as the draft model, block-wise diffusion enables the generation of multiple tokens in a single forward pass, significantly reducing decoding latency and improving hardware utilization. Fig. 1 illustrates the paradigm of speculative decoding based on diffusion models. However, early approaches [20, 32, 31] rely on large diffusion models and exhibit weak coupling with the target model, limiting their practicality. More recently, DFlash [7] achieves the first practically deployable state-of-the-art (SOTA) performance in block diffusion-based speculative decoding. By injecting hidden representations from the target model into the diffusion draft model, it enables high-quality parallel drafting, substantially improving acceptance length and real-world speedup.

Although this paradigm offers strong potential for parallelism, existing methods typically use a fixed block size during inference, directly inherited from the training stage. This design is simple and easy to deploy; however, it overlooks a critical dimension: the decoding policy. Existing methods assume that a single block size is optimal for all inputs and treat it as a static hyperparameter. We argue that this assumption is fundamentally suboptimal. The optimal degree of parallelism depends on input-specific predictability, making block size a sample-dependent decision. Inputs differ in semantic constraints, contextual determinism, and token-level predictability, which lead to varying tolerance for parallel drafting. While larger block sizes improve efficiency for constrained continuations, they may cause error accumulation and lower acceptance for less predictable trajectories; smaller block sizes are more conservative but may underutilize the parallel capacity of diffusion-based generation. Therefore, a fixed block size policy is misaligned with input-level generation characteristics and leaves potential acceleration gains unexplored.

In this paper, we revisit diffusion-based speculative decoding from a largely overlooked perspective. Beyond designing stronger draft models or more efficient verification mechanisms, we ask whether the decoding strategy itself should be treated as a learnable component. We argue that block size is not merely an engineering parameter, but a key control variable that determines the acceptance length in speculative decoding. To validate this insight, we conduct a systematic block size sweep across multiple representative datasets. The results show a clear mismatch between sample-wise optimal block sizes and the fixed configuration used during training. Only a subset of samples achieve optimal performance under the default setting, while many prefer different inference-time block sizes. This suggests that existing fixed strategies fail to fully exploit the efficiency potential of diffusion-based speculative decoding.

Furthermore, we observe that although the optimal block size varies across samples, its distribution is not arbitrarily scattered. Instead, it exhibits a clear local structure: for most samples, the optimal block size concentrates within a narrow region around the training configuration, and few samples achieve optimal performance outside this region. This locality has an important methodological implication. It transforms what would otherwise require expensive online search into a small-scale,

discrete, and well-structured classification problem. In other words, sample-adaptive block size selection does not require complex dynamic optimization, and can instead be efficiently handled by a lightweight predictor.

Based on this insight, we introduce BlockPilot, a sample-adaptive block size selection method. Specifically, after the target model completes prefilling, we use the predictive distribution of the final token as a representation of the current decoding state. Since this token has aggregated full-context information via autoregressive attention, its distribution reflects not only local uncertainty but also the model's estimation of future generation stability, serving as an effective signal for predicting the acceptable block length. Therefore, we formulate block size selection as a policy learning problem over a discrete local action space and approximate the optimal policy with a lightweight predictor. The prediction is performed only once after prefilling and does not modify the target model, draft model, or verification process, allowing seamless integration into existing diffusion-based speculative decoding frameworks. Extensive experiments across models and datasets demonstrate that our method further improves the efficiency of speculative decoding without altering the original inference framework, achieving an acceptance length of 5.92 and a 4.20× speedup on Qwen3-4B at a temperature of 1. Fig. 2 presents the speedup comparison across different methods. Our contributions can be summarized as follows:

- We identify decoding policy as a learnable component in diffusion-based speculative decoding, showing that fixed block-size strategies are suboptimal across diverse inputs.
- We show that the optimal block size follows a structured local distribution, enabling efficient policy learning over a discrete action space.
- We propose BlockPilot, a lightweight instance-adaptive policy learning framework that predicts block size from the prefilling state, achieving consistent speedup gains with minimal overhead.

2 Methodology

2.1 Problem Formulation

Speculative decoding based on diffusion language models [20, 32, 31, 7] is an efficient framework for accelerating autoregressive inference. It trains a lightweight diffusion language model as the draft model with a block size of B, and leverages a block-level diffusion mechanism to generate B tokens in parallel. The target autoregressive model then verifies the generated sequence in parallel, alleviating the sequential bottleneck of autoregressive decoding. Formally, within a single speculative decoding step, the average per-token generation latency is defined as follows [7]:

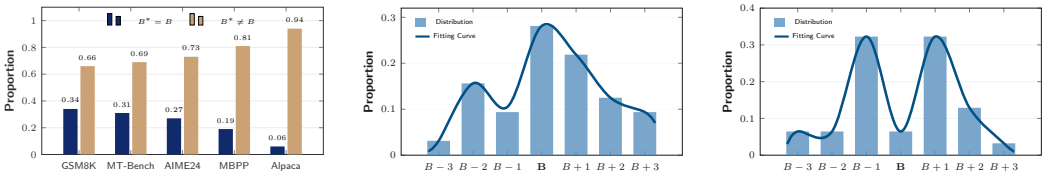
$$L(B) = \frac{T_{\text{draft}}(B) + T_{\text{verify}}(B)}{\tau(B)} \tag{1}$$

where $L(B)$ denotes the average token generation latency under block size B, $T_{\text{draft}}(B)$ and $T_{\text{verify}}(B)$ represent the computational costs of the draft generation and verification stages, respectively, and $\tau(B)$ denotes the expected number of accepted tokens per verification step. Furthermore, the end-to-end speedup $\eta(B)$ is defined as:

$$\eta(B) = \frac{L_{\text{AR}}}{L(B)} \tag{2}$$

where L_{AR} denotes the average latency of standard autoregressive decoding. Since both draft generation and verification can be executed in a block-parallel manner, $T_{\text{draft}}(B)$ and $T_{\text{verify}}(B)$ typically increase sublinearly with B and can often be approximated as near-constant overhead in practice [16, 34, 33]. This implies that $\tau(B)$ primarily governs efficiency, suggesting that an optimal inference block size B^* maximizes $\tau(B)$ and improves end-to-end acceleration.

In common settings, inference typically uses the same block size B as in training to maintain consistency between training and inference. However, this choice may be suboptimal at inference time, since the optimal block size can vary across input samples and deviate from the training configuration. To address this issue, we aim to adaptively determine a sample-specific optimal inference block size B^* for each sample, instead of using a fixed training-time block size B. Specifically, B^* is defined for each sample as the value that maximizes the expected acceptance length during speculative decoding, thereby improving inference efficiency.



(a) Proportion of samples with B^* matching or mismatching B.

(b) Unimodal distribution of B^* peaking at the trained size B.

(c) Symmetric bimodal distribution of B^* near the trained size B.

Figure 3: Analysis of optimal block size B^*. (a) Matching and mismatching proportions across datasets. (b-c) Distribution patterns demonstrating strong locality, where the range $[B-3, B+3]$ covers the optimal size for nearly all samples.

2.2 Key Findings

To systematically analyze the impact of block size on speculative decoding performance, we perform an exhaustive sweep over candidate block sizes on multiple representative datasets. This allows us to directly compare decoding behavior under different levels of block-level parallelism. Specifically, we define the candidate set as $\mathcal{B} = \{1, 2, \ldots, 2B\}$, where B denotes the block size used during training. For each input sample x, we define its optimal block size as follows:

$$B^*(x) = \arg\max_{b \in \mathcal{B}} \tau(b; x) \tag{3}$$

where $\tau(b; x)$ denotes the average number of accepted tokens for sample x under block size b. The acceptance length determines the effective number of generated tokens per decoding step and therefore directly reflects how efficiently the draft tokens are utilized. It thus serves as a key metric for speculative decoding efficiency.

Finding I: Instance-wise Variability of Optimal Block Size. We first observe that the sample-wise optimal block size $B^*(x)$ varies significantly across samples, and does not necessarily match the fixed block size B used during training. As shown in Fig. 3a, we report the proportions of samples satisfying $B^*(x) = B$ and $B^*(x) \neq B$ for each dataset. The results indicate that only a subset of samples align with the training configuration, while a substantial fraction prefer different block sizes at inference time, and this pattern is consistent across datasets.

These observations suggest that a fixed block size is insufficient to capture optimal decoding behavior across diverse inputs. The variation arises from differences in context structure and predictability: for some inputs, strong structural constraints from the prefilling stage enable high consistency over larger blocks, whereas for others, errors accumulate more rapidly as block size increases, resulting in shorter accepted lengths. This mismatch between the fixed training-time block size and sample-specific optimal decoding behavior motivates sample-adaptive block size selection at inference time.

Finding II: Local Interval Property of Optimal Block Size. Although the optimal block size varies across samples, its distribution exhibits a clear local interval structure. As shown in Fig. 3, we observe two dominant patterns in the distribution, both indicating strong locality. Fig. 3b shows a unimodal distribution peaked at the training block size B, with probabilities rapidly decaying on both sides. Fig. 3c instead presents a bimodal but still localized pattern, with both peaks near B. Concretely, for almost all samples, the optimal block size satisfies as follows:

$$B^*(x) \in \{B-k, \ldots, B+k\} \tag{4}$$

and it covers nearly all cases when $k = 3$. Once the block size deviates from B beyond this small range, the acceptance length drops sharply, making such choices rarely optimal. This indicates that the optimal block size exhibits strong locality across samples, with optimal solutions concentrated within a narrow region around the training block size.

This locality has important methodological implications. It reduces the unbounded search space over block sizes to a small discrete interval, within which the optimum almost always lies, significantly simplifying the problem. More importantly, it enables learning-based strategies that select block sizes from a limited candidate set rather than performing global search.

Finding III: Classification Formulation of Block Size Selection. Building on Finding II, which reveals a strong local interval structure of the optimal block size around the training block size, we formulate sample-adaptive block size selection as a structured classification task over a finite discrete space. Specifically, we restrict the candidate set to a local neighborhood $\{B-k, \ldots, B+k\}$ and learn a mapping function that predicts the optimal block size conditioned on the input sample. We use the predictive probability distribution of the last token after the prefilling stage as the input representation. Motivated by the causal structure of autoregressive decoding, the final token attends to the full context and thus provides a globally aggregated summary of the input sequence. Its predictive distribution captures contextual constraints, semantic consistency, and uncertainty in future generation, making it informative for block-size selection. We also explored using only the Top-k probabilities as input, but this led to severe overfitting [15, 4]: the training accuracy reached around 80%, while the test accuracy remained only about 10%. Therefore, we retain the full predictive distribution to preserve richer information and improve generalization. Under this formulation, the probability distribution serves as an approximate sufficient statistic for optimal block size selection, enabling the classifier to make decisions with a single forward pass. Moreover, the proposed module can be seamlessly integrated into existing speculative decoding frameworks without modifying the generation pipeline.

2.3 Training

Following the classification formulation of the block size selection problem, the training objective is to learn a mapping function as follows:

$$f : p(x) \rightarrow B^*(x) \tag{5}$$

where $p(x)$ denotes the predictive probability distribution of the last token after the prefilling stage for input x, and $B^*(x)$ denotes the corresponding optimal block size category. The training process consists of two components: supervised data construction and learning the block size predictor.

Supervised Data Construction. Since the optimal block size is not explicitly annotated, we construct supervised data via an offline enumeration strategy. For each input sample x, we first run the target model's prefilling stage and extract the predictive distribution at the last position $p(x)$ as the input feature. Then, over the local candidate set $\mathcal{B} = \{B-k, \ldots, B+k\}$ determined by Finding II, we enumerate each candidate block size b and execute full speculative decoding to measure the corresponding acceptance length $\tau(b; x)$. The block size that yields the maximum acceptance length is taken as the supervision signal, i.e., the optimal block size $B^*(x)$ defined in Eq. 3. Based on this procedure, we construct the training dataset $\mathcal{D}$ as follows:

$$\mathcal{D} = \{(p(x_i), B^*(x_i))\}_{i=1}^{N} \tag{6}$$

In practice, the candidate set is small (typically $2k+1$), ensuring that the enumeration cost remains tractable. Since evaluations across different candidate block sizes are mutually independent, the process is highly parallelizable. This data construction procedure directly aligns with the optimization objective of speculative decoding, enabling the model to learn a mapping from decoding states to optimal decisions without relying on hand-crafted heuristics.

Block Size Predictor. We adopt a lightweight n-layer multilayer perceptron as a lightweight policy network. Since the input feature $p(x)$ is a high-level state representation compressed from the prefilling stage of the target model, sequence modeling architectures are unnecessary. Instead, we employ a simple discriminative model that is computationally efficient and easy to deploy. The model takes $p(x)$ as input and outputs logits over the candidate block size set $\mathcal{B}$. As $p(x)$ already encodes rich contextual information from the target model, a shallow architecture is sufficient to achieve strong performance while introducing only limited latency. The output distribution is defined via a softmax over candidate block sizes:

$$P(b \mid x) = \frac{\exp(o_b)}{\sum_{b' \in \mathcal{B}} \exp(o_{b'})} \tag{7}$$

where o_b denotes the logit corresponding to the candidate block size $b \in \mathcal{B}$. The model is trained by minimizing the standard cross-entropy loss over the training dataset:

$$L = -\frac{1}{N} \sum_{i=1}^{N} \log P(B^*(x_i) \mid x_i) \tag{8}$$

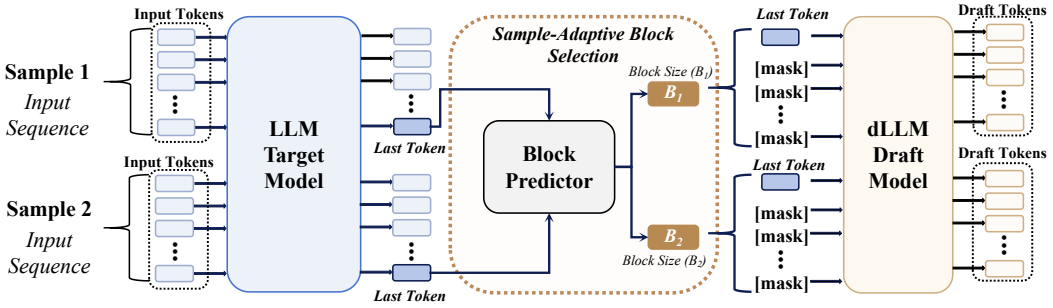


Figure 4: Overview of the BlockPilot inference pipeline. Given an input sequence, the target LLM performs prefilling and produces the predictive distribution of the last token, which serves as a compact representation of the decoding state. This distribution is then fed into a lightweight block size predictor to determine an instance-specific block size. Based on the predicted block size, the diffusion-based draft model generates a block of draft tokens in parallel.

where N denotes the number of training samples and $B^*(x_i)$ represents the ground-truth optimal block size for sample x_i. This objective encourages the model to assign higher probability mass to the optimal block size, thereby learning a direct mapping from decoding states to optimal decisions. Owing to the compact design of the predictor, both training and inference incur relatively low computational overhead.

2.4 Inference

During inference, we integrate the learned block size predictor into the speculative decoding pipeline to enable sample-wise adaptive block size selection. Given an input sample x, we run the prefilling stage of the target model and extract the predictive distribution at the last position, denoted as $p(x)$. The predictor then produces logits over the candidate set $\mathcal{B}$, and the block size is determined as follows:

$$\hat{B}(x) = \arg\max_{b \in \mathcal{B}} f(p(x))_b \tag{9}$$

Table 1: Overhead analysis of the block size predictor compared to backbone models.

Model	#Params	Memory		Latency
		Act.	Param.	
Qwen3-4B	4.41B	6.53 GB	8.62 GB	183.48 ms
Qwen3-8B	8.19B	8.16 GB	16.00 GB	278.37 ms
Llama-3.1-8B-Instruct	8.03B	7.10 GB	15.68 GB	272.14 ms
Block Size Predictor	0.32B	0.02 MB	0.62 GB	7.34 ms

where $f(p(x))_b$ denotes the predicted score for candidate block size b. Once $\hat{B}(x)$ is obtained, it is fixed for the entire subsequent speculative decoding process, including both draft generation and target model verification. Fig. 4 presents an overview of the inference process. Compared to fixed block size strategies, this approach adaptively selects a more suitable level of parallel generation conditioned on each input sample. Notably, the block size prediction is performed only once after the prefilling stage, and the predictor itself is implemented as a lightweight network, resulting in minimal computational overhead. Table 1 compares the predictor with the backbone model in terms of parameter size, memory footprint, and inference latency, showing that the predictor introduces only millisecond-level latency. Although the predictor introduces a small additional memory footprint, this cost is minor compared with the backbone models and is well compensated by the resulting speedup. Therefore, the small additional memory overhead represents a favorable trade-off for improving decoding efficiency.

3 Experiments

3.1 Experimental Setup

Models and Benchmarks. We evaluate our method on four representative LLMs: Qwen3-4B, Qwen3-8B [40], Llama-3.1-8B-Instruct [13], and Qwen3-Coder-30B-A3B [6], spanning diverse scales and domains, including both general-purpose instruction-tuned and code-specialized models. We consider benchmarks across three task categories. For mathematical reasoning, we use GSM8K [12], MATH-500 [24], and AIME24 [26]. For code generation and software engineering,

Table 2: Speedup ratios and average acceptance length τ on Qwen3 models across Math, Code, and Chat benchmarks. Q3-4B and Q3-8B denote Qwen3-4B and Qwen3-8B, respectively. DFlash(n) denotes DFlash with block size n.

Model	Method	Math						Code						Chat		Overall	
		GSM8K		MATH-500		AIME24		HumanEval		MBPP		SWE-Bench		MT-Bench		Avg.	
Temperature = 0		*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ
Q3-4B	EAGLE-3	1.99×	3.30	1.83×	3.08	1.79×	3.05	1.84×	3.05	1.78×	2.95	1.39×	2.48	1.74×	3.02	1.70×	2.95
	DFlash (4)	2.29×	3.32	2.20×	3.48	2.12×	3.45	2.12×	3.22	2.16×	3.17	1.59×	2.58	1.45×	2.85	1.99×	3.15
	DFlash (8)	3.56×	5.16	3.59×	5.68	3.47×	5.64	3.19×	4.84	3.22×	4.71	1.95×	3.17	1.99×	3.91	2.99×	4.73
	DFlash (16)	4.55×	6.59	5.25×	8.31	4.58×	7.45	4.28×	6.51	4.26×	6.24	2.33×	3.78	2.69×	5.29	3.99×	6.31
	DFlash (32)	2.92×	4.24	3.22×	5.09	2.99×	4.86	2.56×	3.89	2.71×	3.97	1.69×	2.74	1.62×	3.19	2.53×	4.00
	BlockPilot	**4.76×**	**6.90**	**5.45×**	**8.62**	**4.80×**	**7.82**	**4.46×**	**6.77**	**4.38×**	**6.42**	**2.45×**	**3.98**	**2.85×**	**5.62**	**4.17×**	**6.59**
Q3-8B	EAGLE-3	1.94×	3.23	1.81×	3.02	1.79×	3.00	1.89×	3.17	1.69×	2.82	1.47×	2.45	1.63×	2.83	1.75×	2.93
	DFlash (4)	2.45×	3.29	2.50×	3.47	2.55×	3.44	2.45×	3.33	2.32×	3.21	1.96×	2.58	1.51×	2.58	2.25×	3.13
	DFlash (8)	3.76×	5.04	4.09×	5.67	4.22×	5.69	3.82×	5.19	3.51×	4.86	2.45×	3.23	2.01×	3.45	3.41×	4.73
	DFlash (16)	4.89×	6.56	5.98×	8.29	5.57×	7.51	4.98×	6.76	4.33×	5.99	2.72×	3.58	2.48×	4.25	4.42×	6.13
	DFlash (32)	4.47×	5.99	5.66×	7.84	5.20×	7.01	4.66×	6.32	3.92×	5.42	2.50×	3.29	2.37×	4.06	4.11×	5.70
	BlockPilot	**5.02×**	**6.73**	**6.17×**	**8.56**	**6.16×**	**8.30**	**5.21×**	**7.07**	**4.53×**	**6.26**	**2.96×**	**3.90**	**2.55×**	**4.37**	**4.66×**	**6.46**
Temperature = 1		*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ
Q3-4B	EAGLE-3	1.89×	3.22	1.75×	2.99	1.64×	2.79	1.74×	3.01	1.69×	2.89	1.47×	2.31	1.77×	2.99	1.70×	2.88
	DFlash (4)	2.32×	3.17	2.34×	3.26	2.08×	2.90	2.29×	3.11	2.25×	3.10	1.79×	2.36	1.62×	2.74	2.10×	2.95
	DFlash (8)	3.60×	4.91	3.71×	5.16	2.97×	4.13	3.37×	4.58	3.21×	4.42	2.17×	2.86	2.24×	3.79	3.04×	4.26
	DFlash (16)	4.36×	5.95	4.83×	6.72	3.66×	5.10	4.32×	5.87	4.02×	5.54	2.33×	3.07	3.10×	5.23	3.80×	5.35
	DFlash (32)	2.83×	3.87	3.35×	4.66	2.79×	3.88	2.76×	3.76	2.69×	3.71	1.91×	2.52	1.91×	3.22	2.61×	3.66
	BlockPilot	**4.77×**	**6.51**	**5.41×**	**7.53**	**4.02×**	**5.60**	**4.74×**	**6.45**	**4.59×**	**6.33**	**2.58×**	**3.39**	**3.31×**	**5.59**	**4.20×**	**5.92**
Q3-8B	EAGLE-3	1.87×	3.12	1.73×	2.91	1.63×	2.74	1.75×	3.05	1.64×	2.74	1.33×	2.11	1.58×	2.70	1.65×	2.77
	DFlash (4)	2.34×	3.17	2.25×	3.22	2.05×	2.92	2.25×	3.05	2.23×	3.03	1.66×	2.20	1.44×	2.45	2.03×	2.86
	DFlash (8)	3.48×	4.72	3.53×	5.06	2.92×	4.17	3.26×	4.41	3.20×	4.36	1.94×	2.57	1.93×	3.28	2.89×	4.08
	DFlash (16)	4.28×	5.81	4.62×	6.62	3.49×	4.98	4.19×	5.68	3.98×	5.41	2.09×	2.77	2.20×	3.75	3.55×	5.00
	DFlash (32)	4.10×	5.57	4.41×	6.32	3.39×	4.84	3.77×	5.11	3.68×	5.00	2.01×	2.67	2.23×	3.79	3.37×	4.75
	BlockPilot	**4.69×**	**6.37**	**5.06×**	**7.25**	**3.84×**	**5.47**	**4.65×**	**6.30**	**4.54×**	**6.17**	**2.30×**	**3.05**	**2.48×**	**4.23**	**3.94×**	**5.55**

we adopt HumanEval [9], MBPP [3], and SWE-Bench [17]. For conversational generation, we use MT-Bench [41]. Together, these benchmarks cover Math, Code, and Chat scenarios.

Baselines. We compare our method with standard autoregressive decoding (baseline), the most classical speculative decoding method with an autoregressive drafter, EAGLE-3 [22], and the state-of-the-art (SOTA) diffusion-based counterpart, DFlash [7]. Here, DFlash(n) denotes DFlash with block size n.

Metrics. Since our method preserves the exact output distribution of the target model under speculative decoding, generation quality remains unchanged. Therefore, we focus on efficiency metrics, measured using the following metrics:

- **Average Acceptance Length τ:** The average number of tokens accepted from the draft model per drafting-verification cycle.
- **Speedup Ratio:** The ratio of inference time for standard autoregressive decoding to that for different speculative decoding methods.

Implementation Details. Our experiments are conducted using the PyTorch framework [29] and the Hugging Face Transformers library [38], running on NVIDIA H100 80GB GPUs. We construct the training dataset from ShareGPT [8], WSC [18], and COPA [30]. We train the model for 100 epochs using the Adam optimizer with a learning rate of $1e^{-5}$. The predictor network consists of two layers with a hidden dimension of 2048, and the default value of the hyperparameter k is set to 2.

3.2 Main Results

Table 2 presents the main results on the Qwen3 series models under both deterministic decoding and stochastic sampling settings. Notably, these improvements are achieved without modifying either the draft or target models and introduce only negligible additional latency. Overall, BlockPilot consistently achieves the best performance across models, temperatures, and benchmark categories.

On Qwen3-4B, it reaches average speedups of 4.17× and 4.20× under temperature $= 0$ and temperature $= 1$, respectively. On Qwen3-8B, it achieves 4.66× and 3.94× under the two settings. These results outperform EAGLE-3 and all fixed-block DFlash variants, demonstrating the effectiveness of sample-adaptive block size selection. This indicates that the proposed adaptive strategy generalizes well across different model scales and decoding regimes.

Compared with the strongest fixed-block DFlash baseline, BlockPilot further improves both speedup and average acceptance length τ. Under temperature $= 0$, DFlash(16) is generally the best fixed-block baseline. On Qwen3-4B, our method improves the average speedup from 3.99× to 4.17× and increases τ from 6.31 to 6.59. On Qwen3-8B, the speedup rises from 4.42× to 4.66×, while τ improves from 6.13 to 6.46. Similar gains are observed under temperature $= 1$, where our method improves the average speedup from 3.80× to 4.20× on Qwen3-4B and from 3.55× to 3.94× on Qwen3-8B. These results reveal the limitation of fixed-block inference: larger blocks provide more parallelism, but do not always yield better acceleration. For example, DFlash(32) often underperforms DFlash(16), suggesting that overly large blocks may reduce acceptance due to accumulated drafting errors. In contrast, our method selects a suitable block size for each input sample, better balancing drafting parallelism and verification acceptance.

Across Math, Code, and Chat benchmarks, our method remains consistently effective. The gains are also preserved under temperature $= 1$, where generation is more stochastic and draft-token acceptance becomes more challenging. This suggests that the last-token predictive distribution after prefilling provides a useful signal for adaptive block size selection. More results are provided in Appendix B.

3.3 Ablation Studies

We conduct ablation studies on Qwen3-4B under a zero-temperature setting to analyze the key design choices of our method, including the predictor architecture, input preprocessing, and candidate interval radius, in order to better understand their impact on performance.

Predictor Configuration. To further understand the impact of predictor design, we report results under different configurations in Table 3. Fixing the depth to $L = 2$, increasing the hidden size from $D = 1024$ to $D = 2048$ improves both speedup and acceptance length τ, while further scaling to $D = 4096$ yields negligible gains. This suggests that a moderate hidden width is sufficient to capture the mapping from prefilling distributions to block size decisions. We then fix the hidden size to $D = 2048$ and vary the depth. Compared to a single-layer predictor, a two-layer model achieves better overall performance, indicating the benefit of moderate nonlinearity. Increasing the depth to $L = 3$ leads to gains on HumanEval but slight drops on GSM8K, yielding no clear overall advantage. Therefore, we adopt a two-layer MLP with hidden size 2048 as the default predictor, striking a good balance between efficiency and performance.

Table 3: Speedup and acceptance length (τ) under different predictor configurations.

Setting	GSM8K		HumanEval		MT-Bench	
	Speedup	τ	*Speedup*	τ	*Speedup*	τ
Hidden width ($L = 2$)						
$D = 1024$	4.73×	6.86	4.41×	6.70	2.85×	5.62
$D = 2048$	4.76×	6.90	4.46×	6.77	2.85×	5.62
$D = 4096$	4.76×	6.90	4.46×	6.77	2.85×	5.62
Predictor depth ($D = 2048$)						
$L = 1$	4.71×	6.83	4.42×	6.72	2.84×	5.59
$L = 2$	4.76×	6.90	4.46×	6.77	2.85×	5.62
$L = 3$	4.65×	6.74	4.67×	7.10	2.85×	5.62

Candidate Interval Radius. We additionally study the effect of the candidate interval radius k in Table 4, where k defines the local block-size search range $B - k, \ldots, B + k$ centered at the training block size B. This hyperparameter balances candidate coverage and prediction difficulty. A smaller radius ($k = 1$) restricts the search space, simplifying prediction but potentially missing better block-size choices. Increasing the radius to $k = 2$ improves both speedup and acceptance length on GSM8K and HumanEval, and also brings gains on MT-Bench, suggesting that a moderately wider interval provides more useful candidates during inference. When further expanded to $k = 3$, performance does not improve on most metrics,

Table 4: Effect of the candidate interval radius k on speedup and acceptance length (τ).

Setting	GSM8K		HumanEval		MT-Bench	
	Speedup	τ	*Speedup*	τ	*Speedup*	τ
$k = 1$	4.67×	6.77	4.44×	6.75	2.78×	5.47
$k = 2$	4.76×	6.90	4.46×	6.77	2.85×	5.62
$k = 3$	4.64×	6.73	4.39×	6.67	2.87×	5.66

likely because the larger candidate set introduces more competing choices and increases prediction difficulty. Therefore, we set $k = 2$ as the default configuration, which achieves a favorable balance between candidate coverage and prediction difficulty.

Predictor Input Preprocessing. We further study how preprocessing the predictor input affects performance in Table 5. The predictor input is the predictive distribution of the last token after prefilling. Compared with directly using this distribution, both normalization and softmax preprocessing consistently reduce speedup and acceptance length τ. This suggests that the raw prefilling distribution already encodes sufficient confidence information for block-size prediction, while additional preprocessing may alter or partially suppress this signal. Therefore, we use the original prefilling distribution as the predictor input.

Table 5: Effect of predictor input preprocessing on speedup and acceptance length (τ).

Setting	GSM8K		HumanEval		MT-Bench	
	Speedup	τ	*Speedup*	τ	*Speedup*	τ
Softmax	4.55×	6.59	4.28×	6.51	2.69×	5.29
Normalization	4.64×	6.73	4.44×	6.75	2.83×	5.58
Ours	**4.76×**	**6.90**	**4.46×**	**6.77**	**2.85×**	**5.62**

4 Related Work

Speculative Decoding. Speculative decoding improves the efficiency of large language model inference by alleviating the inherent sequential constraint of autoregressive generation. Early approaches [19] introduce a lightweight draft model to generate candidate token sequences, which are then verified in parallel by a larger target model. Building on this idea, Medusa [5] eliminates the need for an external draft model by equipping the base LLM with multiple prediction heads, enabling parallel candidate generation via a tree-attention mechanism. More recent work in the EAGLE family [23, 21, 22] further explores feature-level speculative decoding by leveraging intermediate hidden states of the target model. EAGLE-1 predicts future hidden-state distributions to improve token acceptance rates, while EAGLE-2 introduces adaptive drafting structures to better balance efficiency and accuracy. EAGLE-3 further improves training scalability and generalization across model sizes.

Diffusion Language Models. Diffusion-based large language models (dLLMs) offer a non-autoregressive paradigm via parallel masked token prediction. LLaDA [27] first scales dLLMs to the billion-parameter level, achieving performance comparable to LLaMA-3.1-8B [13]. However, fully parallel diffusion models are limited by fixed-length generation and inefficient KV cache usage. Block diffusion models [2] address this by denoising sequences in blocks, combining parallelism with autoregressive structure. Building on this, Fast-dLLM v2 [39] and SDAR [10] convert pretrained autoregressive LLMs into block-diffusion variants, enabling parallel generation with competitive quality on specific tasks. Nevertheless, dLLMs still lag behind state-of-the-art autoregressive models and require many denoising steps, limiting inference efficiency.

Diffusion-based Speculative Decoding. Recent work explores diffusion-based draft generation for speculative decoding. TiDAR [25] combines diffusion and autoregressive objectives for parallel drafting, but still fails to achieve lossless generation quality. Another line of work adapts autoregressive models for diffusion-style drafting. [31] use a LoRA adapter to enable parallel drafting from implicit future-token signals, though with limited effectiveness. DiffuSpec [20] and SpecDiff-2 [32] rely on large pre-trained diffusion LMs with search or alignment to improve acceptance, but require 7B-scale draft models, incurring substantial memory and latency overhead that hinders deployment. All the above methods remain difficult to apply efficiently in practice. Recently, DFlash [7] achieves a practical state-of-the-art (SOTA) method for block diffusion-based speculative decoding by injecting target-model hidden states into the diffusion drafter, substantially improving draft quality, acceptance length, and inference speed.

5 Conclusion

In this paper, we revisit diffusion-based speculative decoding and identify block size as a key factor in inference efficiency. We show that the optimal block size varies across samples but exhibits strong locality around the training configuration, reducing it to a small structured decision problem. Based on this, we propose BlockPilot, a sample-adaptive predictor that uses the prefilling-stage predictive

distribution to select block size from a local candidate set. The method is applied once per sample and integrates seamlessly into existing frameworks. Our method demonstrates that the decoding policy, rather than the model architecture alone, plays a critical role in inference efficiency.

References

[1] Yadagiri Annepaka and Partha Pakray. Large language models: a survey of their development, capabilities, and applications. *Knowledge and Information Systems*, 67(3):2967–3022, 2025.

[2] Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models, 2025. URL https://arxiv.org/abs/2503.09573.

[3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[4] Leonard Berrada, Andrew Zisserman, and M Pawan Kumar. Smooth loss functions for deep top-k classification. *arXiv preprint arXiv:1802.07595*, 2018.

[5] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads, 2024. URL https://arxiv.org/abs/2401.10774.

[6] Ruisheng Cao, Mouxiang Chen, Jiawei Chen, Zeyu Cui, Yunlong Feng, Binyuan Hui, Yuheng Jing, Kaixin Li, Mingze Li, Junyang Lin, et al. Qwen3-coder-next technical report. *arXiv preprint arXiv:2603.00729*, 2026.

[7] Jian Chen, Yesheng Liang, and Zhijian Liu. Dflash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.

[8] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Conghui He, Jiaqi Wang, Feng Zhao, and Dahua Lin. Sharegpt4v: Improving large multi-modal models with better captions. In *European Conference on Computer Vision*, pages 370–387. Springer, 2024.

[9] Mark Chen. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[10] Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, and Bowen Zhou. Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation, 2025. URL https://arxiv.org/abs/2510.06303.

[11] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. *See https://vicuna. lmsys. org (accessed 14 April 2023)*, 2(3):6, 2023.

[12] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[13] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[14] Muhammad Usman Hadi, Rizwan Qureshi, Abbas Shah, Muhammad Irfan, Anas Zafar, Muhammad Bilal Shaikh, Naveed Akhtar, Jia Wu, Seyedali Mirjalili, et al. Large language models: a comprehensive survey of its applications, challenges, limitations, and future prospects. *Authorea preprints*, 1(3):1–26, 2023.

[15] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[16] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Hasan Genc, Kurt Keutzer, Amir Gholami, and Yakun Sophia Shao. Speed: Speculative pipelined execution for efficient decoding. In *Enhancing LLM Performance: Efficacy, Fine-Tuning, and Inference Techniques*, pages 19–32. Springer, 2025.

[17] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

[18] Hector J Levesque, Ernest Davis, and Leora Morgenstern. The winograd schema challenge. *KR*, 2012(13th):3, 2012.

[19] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, pages 19274–19286, 2023.

[20] Guanghao Li, Zhihui Fu, Min Fang, Qibin Zhao, Ming Tang, Chun Yuan, and Jun Wang. Diffuspec: Unlocking diffusion language models for speculative decoding, 2025. URL https://arxiv.org/abs/2510.02358.

[21] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-2: Faster inference of language models with dynamic draft trees, 2024. URL https://arxiv.org/abs/2406.16858.

[22] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test, 2025. URL https://arxiv.org/abs/2503.01840.

[23] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty, 2025. URL https://arxiv.org/abs/2401.15077.

[24] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The twelfth international conference on learning representations*, 2023.

[25] Jingyu Liu, Xin Dong, Zhifan Ye, Rishabh Mehta, Yonggan Fu, Vartika Singh, Jan Kautz, Ce Zhang, and Pavlo Molchanov. Tidar: Think in diffusion, talk in autoregression, 2025. URL https://arxiv.org/abs/2511.08923.

[26] MAA. American Invitational Mathematics Examination - AIME, 2025. URL https://maa.org/math-competitions/american-invitational-mathematics-examination-aime.

[27] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models, 2025. URL https://arxiv.org/abs/2502.09992.

[28] Lorenzo Papa, Paolo Russo, Irene Amerini, and Luping Zhou. A survey on efficient vision transformers: algorithms, techniques, and performance benchmarking. *IEEE transactions on pattern analysis and machine intelligence*, 46(12):7682–7700, 2024.

[29] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[30] Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *AAAI spring symposium: logical formalizations of commonsense reasoning*, pages 90–95, 2011.

[31] Mohammad Samragh, Arnav Kundu, David Harrison, Kumari Nishu, Devang Naik, Minsik Cho, and Mehrdad Farajtabar. Your llm knows the future: Uncovering its multi-token prediction potential, 2025. URL https://arxiv.org/abs/2507.11851.

[32] Jameson Sandler, Jacob K. Christopher, Thomas Hartvigsen, and Ferdinando Fioretto. Specdiff-2: Scaling diffusion drafter alignment for faster speculative decoding, 2025. URL https://arxiv.org/abs/2511.00606.

[33] Andrea Santilli, Silvio Severino, Emilian Postolache, Valentino Maiorca, Michele Mancusi, Riccardo Marin, and Emanuele Rodolà. Accelerating transformer inference for translation via parallel decoding. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12336–12355, 2023.

[34] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31, 2018.

[35] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *ACM Computing Surveys*, 55(6):1–28, 2022.

[36] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[37] Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, et al. Efficient large language models: A survey. *arXiv preprint arXiv:2312.03863*, 2023.

[38] Thomas Wolf. Transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2020.

[39] Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. Fast-dllm v2: Efficient block-diffusion llm, 2025. URL https://arxiv.org/abs/2509.26328.

[40] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[41] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.

A Theoretical Analysis of Sample-Adaptive Block Size Selection

A.1 Acceptance Length as a Prefix-Survival Process

Prefix-survival identity. We first formalize the expected acceptance length in speculative decoding from a prefix-survival perspective. Given an input sequence x and an inference block size b, let $L_b(x)$ denote the number of consecutive draft tokens accepted by the target model in one speculative decoding step. The expected acceptance length is defined as

$$\tau(b; x) = \mathbb{E}[L_b(x)]. \tag{10}$$

Since $L_b(x)$ is a non-negative integer-valued random variable bounded by the block size b, its expectation can be decomposed into the sum of survival probabilities:

$$\tau(b; x) = \sum_{i=1}^{b} \mathbb{P}(L_b(x) \geq i). \tag{11}$$

This identity provides a prefix-level view of speculative decoding: each term measures the probability that the verified prefix survives up to at least position i.

In speculative decoding, the target model accepts only the longest consistent prefix of the drafted block. Therefore, accepting at least i draft tokens requires that the first i drafted tokens are all accepted. Define the prefix-consistency event

$$A_i = \{\text{the first } i \text{ drafted tokens are all accepted}\}. \tag{12}$$

Then

$$\mathbb{P}(L_b(x) \geq i) = \mathbb{P}(A_i). \tag{13}$$

We define the conditional acceptance probability at position j as

$$q_j(x, b) = \mathbb{P}\left(\text{the } j\text{-th drafted token is accepted} \mid L_b(x) \geq j - 1, x, b\right). \tag{14}$$

By the chain rule of probability, the probability that the first i drafted tokens all survive verification is

$$\mathbb{P}(A_i) = \prod_{j=1}^{i} q_j(x, b). \tag{15}$$

Substituting this expression into the survival identity yields

$$\tau(b; x) = \sum_{i=1}^{b} \prod_{j=1}^{i} q_j(x, b). \tag{16}$$

Equation (16) shows that the acceptance length can be interpreted as the expected stopping time of a truncated prefix-survival process. This formulation is more informative than treating acceptance length as a black-box empirical statistic: the block size determines the truncation horizon, while the actual contribution of each additional drafted position is governed by the corresponding prefix survival probability.

Block-size-dependent acceptance. The conditional probability $q_j(x, b)$ should not be viewed as an intrinsic constant independent of the inference strategy. In block-level diffusion drafting, the draft model generates a set of mutually dependent tokens within a single block. Increasing the block size enlarges the maximum possible accepted length, but it may also make each prefix harder to verify because the proposal distribution must remain coherent over a longer span.

Therefore, increasing b induces two competing effects:

$$\tau(b; x) = \sum_{i=1}^{b} \underbrace{\prod_{j=1}^{i} q_j(x, b)}_{\text{prefix survival probability}}. \tag{17}$$

A larger block size increases the summation horizon by adding more candidate positions. However, the same change may alter the conditional acceptance probabilities inside each multiplicative term. Since prefix survival is multiplicative, even a mild degradation in $q_j(x, b)$ can be amplified over longer prefixes. Hence, the expected acceptance length is not determined solely by the number of drafted tokens, but by the interaction between block length and prefix survival.

A.2 Locality Induced by Predictability and Block-Size Retention

Predictability–retention decomposition. Let B denote the block size used to train the diffusion draft model. Since the draft model is optimized to generate coherent token blocks under this configuration, its proposal distribution is naturally best calibrated near B. When the inference block size b deviates substantially from B, the draft distribution may become less aligned with the target model, which can reduce proposal quality and lower acceptance probabilities.

To capture this mechanism in a simple analytical form, we decompose the conditional acceptance probability as

$$q_j(x, b) = \gamma_j(x)\, r(b; B), \tag{18}$$

where $\gamma_j(x) \in (0, 1]$ represents the intrinsic predictability of sample x at position j, and $r(b; B) \in (0, 1]$ measures the retention of proposal quality under inference block size b relative to the training block size B. This decomposition separates two effects: sample-dependent predictability and block-size-induced proposal degradation.

A convenient retention model is

$$r(b; B) = \exp\{-\alpha(b - B)^2\}, \quad \alpha > 0. \tag{19}$$

This function reaches its maximum at the training block size B and gradually penalizes deviations from it. We emphasize that this retention function is used only as an analytical model to explain the observed locality of optimal block sizes, rather than as an additional component required by the algorithm.

Substituting the decomposition into Eq. (16), we obtain

$$\tau(b; x) = \sum_{i=1}^{b} r(b; B)^i \prod_{j=1}^{i} \gamma_j(x). \tag{20}$$

This expression makes the local structure explicit. Although increasing b adds more candidate positions, deviations from B reduce the prefix survival terms through the factor $r(b; B)^i$. The effect becomes stronger for longer prefixes because the retention factor is exponentiated by the prefix length.

Geometric approximation. For interpretation, suppose that the intrinsic predictability is approximately stable across positions:

$$\gamma_j(x) \approx \gamma_x, \quad 0 < \gamma_x < 1. \tag{21}$$

Define the effective survival factor

$$\rho_x(b) = \gamma_x r(b; B). \tag{22}$$

Since $0 < \gamma_x < 1$ and $0 < r(b; B) \leq 1$, we have

$$0 < \rho_x(b) < 1. \tag{23}$$

Then Eq. (20) reduces to

$$\tau(b; x) \approx \sum_{i=1}^{b} \rho_x(b)^i. \tag{24}$$

Using the finite geometric series identity, we obtain

$$\tau(b; x) \approx \frac{\rho_x(b)\left[1 - \rho_x(b)^b\right]}{1 - \rho_x(b)}. \tag{25}$$

Equation (25) explains the mechanism behind sample-adaptive block-size selection. The factor γ_x captures sample-level predictability: structured or deterministic inputs tend to have larger γ_x and can sustain longer verified prefixes, whereas uncertain or open-ended inputs have smaller γ_x and experience faster survival decay. Meanwhile, $r(b; B)$ captures block-size-induced proposal degradation and discourages choices far from the training configuration. Therefore, the optimal block size is governed by the interaction between sample predictability and block-size retention.

Local candidate interval. For a candidate set $\mathcal{B}$, the sample-wise optimal block size is defined as

$$B^*(x) = \arg\max_{b \in \mathcal{B}} \tau(b; x). \tag{26}$$

Under the retention model above, block sizes far from B are penalized through $r(b; B)^i$, especially for longer prefixes. This provides a theoretical rationale for restricting the candidate space to a local interval around the training block size:

$$\mathcal{B}_{\text{loc}} = \{b \in \mathcal{B} : |b - B| \le k\}. \tag{27}$$

Empirically, this local interval captures the optimal block size for most samples. The resulting conclusion is that the best block size is sample-dependent, but it is expected to concentrate near the block size used to train the diffusion draft model. This locality reduces the adaptive block-size selection problem from an expensive global search to a structured local decision problem.

A.3 Regret of Local Block-Size Prediction

Acceptance-length regret. We finally analyze how prediction error affects the acceptance length. Let $B^*(x)$ be the optimal block size defined over the candidate set $\mathcal{B}$, and let $\hat{B}(x) \in \mathcal{B}$ be the block size predicted by the learned predictor. The acceptance-length regret is defined as

$$R(x) = \tau(B^*(x); x) - \tau(\hat{B}(x); x). \tag{28}$$

Since block size is selected from a discrete candidate set, we assume a discrete Lipschitz condition over $\mathcal{B}$:

$$|\tau(b_1; x) - \tau(b_2; x)| \le L_\tau |b_1 - b_2|, \quad \forall b_1, b_2 \in \mathcal{B}, \tag{29}$$

for some constant $L_\tau > 0$. Then

$$\begin{aligned} R(x) &= \tau(B^*(x); x) - \tau(\hat{B}(x); x) \\ &\le \left| \tau(B^*(x); x) - \tau(\hat{B}(x); x) \right| \\ &\le L_\tau |B^*(x) - \hat{B}(x)|. \end{aligned} \tag{30}$$

Taking expectation over the data distribution gives

$$\mathbb{E}[R(x)] \le L_\tau \mathbb{E}\left[|B^*(x) - \hat{B}(x)| \right]. \tag{31}$$

This bound shows that the loss in acceptance length is controlled by the distance between the predicted and optimal block sizes. Exact prediction is therefore not strictly necessary: if the predictor selects a block size close to the optimum, the regret remains bounded. This supports two design choices of our method. First, the prediction problem can be restricted to a local candidate set around B, where neighboring block sizes have similar acceptance behavior. Second, the predictor can be lightweight, since it does not need to solve a global optimization problem, but only needs to identify a near-optimal block size within a structured local interval.

B Performance Evaluation on Instruction and Code Models

Table 6 reports results on Llama-3.1-8B-Instruct and Qwen3-Coder-30B-A3B across Math, Code, and Chat benchmarks. Overall, our method consistently achieves the best performance across both models and decoding settings.

Under temperature $= 0$, our method attains the highest average speedup on both Llama and Qwen, reaching 3.25× and 4.12×, respectively, while also achieving the best or near-best average acceptance length τ. In particular, on Qwen, it significantly outperforms all DFlash variants, improving speedup from 3.86× (DFlash(16)) to 4.12×, with consistent gains in τ. Under temperature $= 1$, similar trends hold. Our method achieves 2.40× and 3.95× average speedups on Llama and Qwen, respectively, outperforming all baselines across most benchmarks. Notably, even under higher sampling uncertainty, it maintains competitive or superior acceptance lengths, indicating stable draft quality.

Across task categories, the improvements are consistent on Math, Code, and Chat benchmarks, suggesting that the benefits of adaptive block selection generalize across heterogeneous workloads and model scales. Overall, these results further confirm the effectiveness and robustness of the proposed method across both medium-scale and large-scale LLMs.

Table 6: Speedup ratios and average acceptance length τ on Llama and Qwen models across Math, Code, and Chat benchmarks. Llama denotes Llama-3.1-8B-Instruct, and Qwen denotes Qwen3-Coder-30B-A3B. DFlash(n) denotes DFlash with block size n.

Model	Method	Math						Code						Chat		Overall	
		GSM8K		MATH-500		AIME24		HumanEval		MBPP		SWE-Bench		MT-Bench		Avg.	
Temperature $= 0$		*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ
Llama	EAGLE-3	1.81×	3.06	1.70×	2.98	1.64×	2.99	1.94×	3.25	1.87×	3.22	1.60×	2.80	1.50×	2.72	1.72×	3.00
	DFlash (4)	2.21×	3.16	2.07×	3.08	2.00×	3.09	2.36×	3.35	2.28×	3.32	1.95×	2.90	1.83×	2.82	2.10×	3.10
	DFlash (8)	2.89×	4.13	2.93×	4.37	3.03×	4.69	3.42×	4.86	3.39×	4.94	2.51×	3.74	2.33×	3.59	2.93×	4.33
	DFlash (16)	2.91×	4.16	3.11×	4.63	2.93×	4.54	3.67×	5.21	3.57×	5.20	2.52×	3.76	2.52×	3.88	3.03×	4.48
	DFlash (32)	2.87×	4.09	2.79×	4.16	2.80×	4.33	3.59×	5.09	3.57×	5.20	2.51×	3.74	2.66×	4.09	2.97×	4.39
	BlockPilot	**2.96×**	**4.22**	**3.21×**	**4.79**	**3.63×**	**5.62**	**3.74×**	**5.31**	**3.75×**	**5.46**	**2.71×**	**4.04**	**2.78×**	**4.28**	**3.25×**	**4.82**
Qwen	EAGLE-3	1.75×	2.94	1.85×	3.07	1.80×	3.00	1.92×	3.41	1.89×	3.31	1.47×	2.53	1.21×	2.34	1.70×	2.94
	DFlash (4)	2.13×	3.04	2.26×	3.17	2.19×	3.10	2.34×	3.51	2.30×	3.41	1.79×	2.63	1.48×	2.44	2.07×	3.04
	DFlash (8)	2.94×	4.20	3.34×	4.70	3.04×	4.32	3.86×	5.79	3.69×	5.47	2.21×	3.24	2.01×	3.32	3.01×	4.43
	DFlash (16)	3.58×	5.12	4.16×	5.85	3.67×	5.20	5.74×	8.61	5.11×	7.58	2.48×	3.65	2.29×	3.77	3.86×	5.68
	DFlash (32)	3.38×	4.83	3.91×	5.49	3.43×	4.86	5.60×	8.40	4.87×	7.22	2.35×	3.46	2.21×	3.64	3.68×	5.42
	BlockPilot	**3.75×**	**5.36**	**4.37×**	**6.14**	**4.19×**	**5.95**	**6.05×**	**9.07**	**5.26×**	**7.80**	**2.70×**	**3.96**	**2.49×**	**4.10**	**4.12×**	**6.05**
Temperature $= 1$		*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ	*Speedup*	τ
Llama	EAGLE-3	1.51×	2.79	1.19×	2.28	0.76×	1.35	1.74×	2.95	1.72×	2.94	1.24×	2.20	0.92×	1.76	1.30×	2.32
	DFlash (4)	1.84×	2.89	1.45×	2.38	0.93×	1.45	2.12×	3.05	2.10×	3.04	1.51×	2.30	1.12×	1.86	1.58×	2.42
	DFlash (8)	2.16×	3.40	1.67×	2.75	1.11×	1.74	2.94×	4.22	2.92×	4.24	1.47×	2.24	1.42×	2.36	1.96×	2.99
	DFlash (16)	2.45×	3.85	1.75×	2.88	1.15×	1.80	3.00×	4.31	2.80×	4.06	1.60×	2.44	1.54×	2.56	2.04×	3.13
	DFlash (32)	1.76×	2.76	1.65×	2.71	0.99×	1.55	2.85×	4.10	3.05×	4.42	1.52×	2.32	1.54×	2.56	1.91×	2.92
	BlockPilot	**2.73×**	**4.29**	**2.10×**	**3.45**	**1.48×**	**2.32**	**3.29×**	**4.72**	**3.37×**	**4.88**	**1.89×**	**2.88**	**1.92×**	**3.20**	**2.40×**	**3.68**
Qwen	EAGLE-3	1.69×	2.83	1.71×	2.96	1.59×	2.74	1.91×	3.34	1.82×	3.20	1.34×	2.27	1.20×	2.27	1.61×	2.80
	DFlash (4)	2.06×	2.93	2.09×	3.06	1.94×	2.84	2.33×	3.44	2.22×	3.30	1.63×	2.37	1.46×	2.37	1.96×	2.90
	DFlash (8)	2.89×	4.10	3.05×	4.47	2.56×	3.76	3.77×	5.58	3.67×	5.44	1.96×	2.86	2.01×	3.28	2.84×	4.21
	DFlash (16)	3.41×	4.84	3.72×	5.45	3.09×	4.54	5.27×	7.79	4.91×	7.28	2.09×	3.04	2.27×	3.70	3.54×	5.23
	DFlash (32)	3.41×	4.84	3.61×	5.29	2.88×	4.22	5.54×	8.19	4.89×	7.25	2.09×	3.05	2.08×	3.38	3.50×	5.17
	BlockPilot	**3.77×**	**5.36**	**4.01×**	**5.87**	**3.33×**	**4.89**	**6.35×**	**9.39**	**5.34×**	**7.92**	**2.36×**	**3.44**	**2.48×**	**4.04**	**3.95×**	**5.84**

C Limitations and Future Work

In this paper, our data construction method provides an effective way to identify suitable block sizes for different training samples, enabling the model to learn more fine-grained execution preferences. However, this benefit comes with some computational burden, particularly for very large models. For example, for a 32B model, executing a single sample under one block size takes approximately 5 seconds. To construct one training sample, we evaluate all candidate block sizes in the range $\{B-k, \ldots, B+k\}$. Under our default setting of $k=2$, this requires five executions with different block sizes, resulting in roughly 25 seconds per training sample. Nevertheless, this overhead is incurred only once during data construction and can be performed entirely offline, without affecting the efficiency of model training or inference. Although this cost is acceptable in our setting, it could be further reduced by more efficient search strategies. Since optimizing the data construction pipeline is not the main focus of this work, we leave this direction to future work. Possible improvements include heuristic search strategies, adaptive block-size pruning, and early stopping mechanisms to avoid evaluating unnecessary candidates. In addition, one could explore lightweight proxy metrics to estimate the effectiveness of different block sizes before full execution, or adopt a coarse-to-fine search strategy that first evaluates a small set of representative candidates and then refines the search around promising regions. Another possible direction is to reuse execution results across similar samples, thereby reducing redundant evaluations during data construction.